OPERATING SYSTEMS MINI PROJECT · SINGLE-CORE CPU SCHEDULING

CPU Scheduler Lab

Project Workflow & Algorithm Guide

From a list of processes to a Gantt chart, per-process metrics and an algorithm comparison. This document explains what the project does, how the scheduler assigns the CPU, what each algorithm selects, the inputs and outputs, and every metric formula used in the code.

Flask | Python 3 | HTML / CSS / JavaScript | Chart.js | 6 algorithms

PROJECT TYPE	ALGORITHMS	SCHEDULING SCOPE	AUTOMATED TESTS
Browser simulator	**Six**	**Single core**	**57 passing**

Simulation only. This project demonstrates CPU scheduling algorithms. It does not read, control or modify the real scheduler of your operating system.

CONTENTS & OVERVIEW

What this project does

A teaching simulator for the short-term CPU scheduler.

01

The project is a complete browser-based CPU scheduling simulator. The user builds a process list, selects one of six algorithms, sets a time quantum for Round Robin, and runs the simulation. A Flask backend validates the request, executes the chosen algorithm in pure Python, calculates the performance metrics, and returns JSON. The browser then draws a custom Gantt chart, a per-process metrics table, summary cards and—on request—a six-algorithm comparison with Chart.js bar charts.

6	1	57	8
ALGORITHMS	CPU CORE	TESTS PASSING	WORKFLOW STEPS

WHAT THE USER CAN DO

- Add, edit and delete processes with PID, arrival, burst and priority.
- Load a five-process sample or clear the list.
- Run any single algorithm and inspect the schedule.
- Compare all six algorithms on identical inputs.
- Read CT, TAT, WT, RT, averages and CPU utilization.
- Hover or focus Gantt blocks for start and end times.

WHAT THE SYSTEM GUARANTEES

- Consistent tie-breaking: primary key, arrival, then PID.
- Simultaneous arrivals become available together.
- Idle CPU time is shown as an `Idle` block.
- Round Robin follows the arrival-at-expiry queue rule.
- Every algorithm uses the same metric formulas.
- Invalid input is rejected with a clear inline message.

Contents

01 **Overview**
Purpose, scope and the teaching model

02 **End-to-end workflow**
Browser, Flask, scheduling engine and rendering

03 **How tasks are assigned**
Ready set, decision points, tie-breaks and idle time

04 **The six algorithms**
Selection rules, pseudocode and trade-offs

05 **Inputs and validation**
Process fields, quantum, accepted ranges and errors

06 **Outputs and formulas**
Gantt data, per-process rows, CT/TAT/WT/RT/utilization

07 **Worked example**
Five processes, six schedules and a comparison

08 **Implementation & verification**
Files, API, tests, limitations and viva summary

The central idea: a scheduling algorithm is a repeated choice. At each decision point, the project builds the set of arrived and unfinished processes, applies the algorithm's key, runs the winner, records the interval, and repeats. All metrics are derived from those intervals and completion times.

WORKFLOW

From process list to Gantt chart

The browser never calculates the schedule; Flask and the pure-Python scheduler do.

02

1	**Build the process list** Add PID, arrival time, burst time and priority. Edit or delete rows at any time.	BROWSER
2	**Choose an algorithm and quantum** Select one of six policies. The quantum field appears only for Round Robin.	BROWSER
3	**Run simulation** The full list is sent as JSON to `/api/schedule` or `/api/compare`.	HTTP
4	**Validate the request** Flask checks fields, ranges, duplicate IDs, reserved `Idle`, quantum and algorithm key.	FLASK
5	**Schedule the processes** The chosen algorithm produces timer intervals and first-start/completion times.	PURE PYTHON
6	**Calculate metrics** CT, TAT, WT, RT, averages and CPU utilization are derived in one shared module.	PURE PYTHON
7	**Return JSON** The response contains the timeline, process rows and metrics; comparison adds every algorithm and the winners.	FLASK
8	**Render the result** `gantt.js` draws the timeline; `charts.js` draws the comparison bars.	BROWSER

Presentation

HTML structure, responsive CSS, accessible forms, tables and empty states.

→

Application

Flask routes, JSON validation, error responses and the pure-Python scheduler.

→

Visualisation

Custom Gantt blocks plus Chart.js bar charts for average WT, TAT and RT.

SINGLE RUN REQUEST

```
POST /api/schedule
{
  "processes": [
    {"pid": "P1", "arrival": 0,
     "burst": 4, "priority": 2}
  ],
  "algorithm": "srtf",
  "quantum": null
}
```

The quantum is required only for `rr`. SRTF/Priority use only the process fields.

WHAT COMES BACK

- `timeline` — ordered Gantt blocks, including Idle.
- `processes` — start, completion and metrics per process.
- `metrics` — averages, utilization, busy/idle/total.
- `algorithm` — display name and quantum.

Comparison additionally returns `results` for all six and a `best` map.

Stateless design: the server stores nothing. Every request carries the entire process list, so two browser tabs cannot interfere and refreshing the page simply clears the local list.

SCHEDULING LOGIC

How the CPU assigns tasks

This is a simulation of a short-term scheduler, not the real OS scheduler.

03

Each process record in the browser is a simplified PCB: PID, arrival time, burst time, priority and the derived scheduling fields. The scheduler keeps a **ready set** of processes that have arrived and still need CPU time. At each decision point it selects one process, runs it for an interval, records a Gantt block, and updates the remaining work.

DECISION LOOP

1. **Build available set**
   `arrival ≤ now` and remaining burst > 0.
2. **Choose by policy**
   Apply the algorithm's primary key, then arrival, then PID.
3. **Run an interval**
   To completion, to the next arrival, or for one quantum.
4. **Record and update**
   Append a block, reduce remaining time, record completion.
5. **Repeat or idle**
   If nobody is ready, add an Idle block and jump to the next arrival.

WHEN DOES THE SCHEDULER CHOOSE?

ALGORITHM STYLE	DECISION POINTS
Non-preemptive FCFS, SJF, Priority	When the CPU becomes idle: at the start, or after a process completes.
Preemptive SRTF, Preemptive Priority	When a new process arrives, plus when the CPU becomes idle.
Round Robin	When a quantum expires, when the CPU becomes idle, and when a new arrival enters the ready queue.

Selection keys and tie-breaking

ALGORITHM	PRIMARY KEY	SECOND	FINAL TIE-BREAK	PREEMPTIVE?
FCFS	Arrival time	Process ID	—	No
SJF	Burst time	Arrival time	Process ID	No
SRTF	Remaining time	Arrival time	Process ID	Yes
Round Robin	Queue order	Arrival/PID on entry	—	Yes (quantum)
Priority	Priority number	Arrival time	Process ID	No
Preemptive Priority	Priority number	Arrival time	Process ID	Yes

Lower priority number = higher priority. A new process preempts only when its priority number is *strictly smaller* than the running process's priority. Equal priority keeps the earlier arrival on the CPU. Round Robin arriving exactly at quantum expiry enters the ready queue before the preempted process is re-added.

SIMULTANEOUS ARRIVALS

All processes with the same arrival time are available together and ordered by the same tie-breaking keys.

IDLE CPU

If the available set is empty, an `Idle` block is added and time jumps to the next arrival.

GANTT MERGING

Adjacent blocks of the same process are merged, especially after Round Robin slices or no-preemption arrivals.

ALGORITHM REFERENCE

The six scheduling algorithms

Same process data, six different CPU assignment policies.

04

ALGORITHM	BEST KNOWN FOR	MAIN RISK	KEY IDEA
FCFS	Simplicity and fairness in arrival order	Convoy effect	Earliest arrival runs to completion.
SJF	Low average waiting/turnaround in batches	Starvation; burst unknown in practice	Shortest arrived job runs to completion.
SRTF	Short-job response and turnaround	Starvation; more switching	Shortest remaining time can preempt.
Round Robin	Fairness and response time	More context switches; quantum tuning	Fixed quantum, FIFO ready queue.
Priority	Important processes first	Starvation of low priority	Smallest priority number runs to completion.
Preemptive Priority	Immediate response to critical arrivals	Starvation; more switching	Higher-priority arrival can preempt.

01 First Come, First Served (FCFS)

NON-PREEMPTIVE

Processes are ordered by arrival time. The earliest arrival runs until it finishes. If two processes arrive together, the smaller Process ID runs first. This is the simplest policy and the baseline for comparison.

SELECTION

Minimum (`arrival, PID`); no later arrival can interrupt.

TIE-BREAK

Earlier arrival, then smaller PID (natural order: P2 before P10).

```
sort processes by (arrival, PID)
time = 0
for each process:
    if time < arrival: add Idle block; time = arrival
    run process from time to time + burst
    record start and completion
```

+ Trivial to implement; no starvation; low overhead.

− Convoy effect: short jobs wait behind one long job.

02 Shortest Job First (SJF)

NON-PREEMPTIVE

Among the processes that have already arrived, the smallest burst time runs next and completes. SJF usually lowers average waiting and turnaround time because short jobs do not wait behind long ones.

SELECTION

Minimum (`burst, arrival, PID`) among available processes.

TIE-BREAK

Equal burst → earlier arrival → smaller PID.

```
pending = all processes; time = 0
while pending:
    available = {p | p.arrival <= time}
    if available is empty: idle to next arrival
    chosen = min(available, key = (burst, arrival, PID))
    run chosen to completion; record it
```

+ Good average TAT/WT for known batch workloads.

− Long jobs can starve; real burst times are estimates.

SRTF and Round Robin

Shortest-remaining-time preemption versus fixed time slicing.

04

03 Shortest Remaining Time First (SRTF)

PREEMPTIVE

SRTF is preemptive SJF. At every arrival it compares the remaining burst times of all available processes. A new process with a strictly shorter remaining time preempts the running process immediately.

SELECTION

Minimum `(remaining, arrival, PID)`; re-evaluated at every arrival.

TIE-BREAK

Equal remaining → earlier arrival → smaller PID. Equal remaining does not preempt.

```
remaining = burst for every process; time = 0
while unfinished:
    available = {p | arrived and remaining[p] > 0}
    if empty: idle to next arrival
    chosen = min(available, key = (remaining, arrival, PID))
    run until completion or next arrival, whichever is first
    update remaining; record block and completion
```

+ Short jobs get CPU quickly; can lower average TAT/WT.

− Long jobs may starve; more preemptions than SJF.

04 Round Robin (RR)

PREEMPTIVE

Each ready process runs for at most one time quantum. If it does not finish, it goes to the back of the FIFO ready queue. New arrivals enter the queue in arrival/PID order; a process arriving exactly when a quantum expires enters before the preempted process is re-added.

SELECTION

Leftmost process in the ready queue; run `min(quantum, remaining)`.

TIE-BREAK

Queue order on entry: earlier arrival, then smaller PID.

```
ready = empty FIFO queue; time = 0
enqueue arrivals up to time
while unfinished:
    if ready empty: idle to next arrival; enqueue arrivals
    p = ready.popleft()
    run p for min(quantum, remaining[p])
    enqueue new arrivals at the new time
    if p unfinished: ready.append(p)   # preempted process goes last
    else: record completion
```

+ Fair; good response time; no starvation for finite bursts.

− More switches; a very large quantum behaves like FCFS.

The two priority policies

Same importance rule; one waits, one can preempt.

04

05 Priority Scheduling

NON-PREEMPTIVE

Among arrived processes, the smallest priority number runs to completion. This is useful when important work must go first, but a continuously arriving stream of high-priority work can starve low-priority processes. Aging is not implemented.

SELECTION

Minimum `(priority, arrival, PID)` among available processes.

TIE-BREAK

Equal priority → earlier arrival → smaller PID.

```
pending = all processes; time = 0
while pending:
    available = {p | p.arrival <= time}
    if empty: idle to next arrival
    chosen = min(available, key = (priority, arrival, PID))
    run chosen to completion; record it
```

+ Models importance; simple non-preemptive variant.

− Low-priority starvation; priority must be assigned sensibly.

06 Preemptive Priority

PREEMPTIVE

The running process is interrupted when a newly arrived process has a strictly smaller priority number. Equal priority keeps the earlier-arrived process running. SRTF and Preemptive Priority share the same `_preemptive()` engine; only the selection key differs.

SELECTION

Minimum `(priority, arrival, PID)`; re-evaluated at every arrival.

TIE-BREAK

Equal priority → earlier arrival → smaller PID. Equal priority does not preempt.

```
remaining = burst for every process; time = 0
while unfinished:
    available = {p | arrived and remaining[p] > 0}
    if empty: idle to next arrival
    chosen = min(available, key = (priority, arrival, PID))
    run until completion or next arrival, whichever is first
    update remaining; record block and completion
```

+ Critical work gets immediate CPU; improves response time.

− Starvation; more preemptions than non-preemptive Priority.

Shared implementation: SJF and non-preemptive Priority share `_non_preemptive()`; SRTF and Preemptive Priority share `_preemptive()`. Round Robin is independent because it uses a fixed quantum and a FIFO queue.

INPUTS

What the user provides

Every value is validated on the client for feedback and again on the server for correctness.

05

INPUT	TYPE / RULE	EXAMPLE	WHY IT MATTERS
Process ID	1–16 letters, digits, _ or -; unique, case-insensitive; `Idle` reserved	`P1`	Identifies rows and Gantt blocks; natural ordering makes P2 come before P10.
Arrival time	Whole number, ≥ 0	`0`	Determines when the process enters the ready set; creates initial idle gaps.
Burst time	Whole number, > 0	`4`	Total CPU work; also the initial remaining time in preemptive algorithms.
Priority	Whole number, ≥ 0	`2`	Used by both Priority policies; lower number = higher priority.
Algorithm	One of six keys	`srtf`	Selects the scheduling policy and response shape.
Quantum	Whole number, > 0; required only for `rr`	`2`	Maximum continuous CPU time per Round Robin turn.

SAMPLE WORKLOAD

PID	ARRIVAL	BURST	PRIORITY
P1	0	4	2
P2	1	3	1
P3	2	1	3
P4	3	2	2
P5	4	5	4

Round Robin quantum = 2. Loaded by the “Load sample” button.

REJECTED INPUTS

- Empty process list, or more than 100 processes.
- Empty, invalid, reserved or duplicate Process ID.
- Negative arrival or priority.
- Burst time of 0 or less.
- Decimal or non-numeric time values.
- Missing quantum, or quantum ≤ 0 for Round Robin.
- Unknown algorithm key or malformed JSON.

Errors appear inline, not in browser alerts.

All times are whole-number units. This keeps the Gantt chart and formulas exact. Decimal values such as `2.5` are rejected rather than rounded silently. The maximum list size is 100 processes.

OUTPUTS & FORMULAS

What the simulator returns

A timeline, one row per process, and a small metrics object.

06

GANTT TIMELINE

A chronological list of `{pid, start, end}` blocks. `Idle` marks no ready process; adjacent same-process blocks are merged.

PER-PROCESS ROWS

PID, arrival, burst, priority, first start, completion, TAT, WT and RT. Returned in the user's input order.

SUMMARY METRICS

Average TAT/WT/RT, CPU utilization, process count, busy time, idle time and total elapsed time.

Definitions and formulas

COMPLETION TIME

CT_i

Time at which process i finishes its burst.

TURNAROUND TIME

$TAT_i = CT_i - AT_i$

Total time from arrival to completion.

WAITING TIME

$WT_i = TAT_i - BT_i$

Time spent waiting, excluding its own execution.

RESPONSE TIME

$RT_i = first_start_i - AT_i$

Delay before the first CPU allocation.

AVERAGE METRIC

$avg = (1/n)\ \Sigma\ value_i$

Applied to TAT, WT and RT.

CPU UTILIZATION

$(busy \div elapsed) \times 100$

Busy blocks only; elapsed is 0 to the last completion.

WORKED FCFS CALCULATION

PID	AT	BT	CT	TAT	WT	RT
P1	0	4	4	4	0	0
P2	1	3	7	6	3	3
P3	2	1	8	6	5	5
P4	3	2	10	7	5	5
P5	4	5	15	11	6	6

Average TAT = 34/5 = **6.80** · WT = 19/5 = **3.80** · RT = **3.80** · busy 15 / elapsed 15 = **100%**.

READING THE OUTPUT

- **CT** is the end of the process's last block.
- **TAT** includes waiting and execution.
- **WT** is zero if the process starts immediately.
- **RT** stops at the first start; it can differ from WT after preemption.
- **Busy** is the sum of non-idle block durations.
- **Total elapsed** includes initial idle time from t = 0.
- **Best** means minimum average TAT/WT/RT and maximum CPU utilization; ties are preserved.

Non-preemptive shortcut: if a process runs once to completion, its first start equals completion minus burst, so `RT = WT`. In SRTF, Preemptive Priority and Round Robin, the two can differ because a process may be interrupted and resume later.

WORKED EXAMPLE

One workload, six schedules

The sample workload with Round Robin quantum = 2. Metrics are from the implemented code.

07

P1 P2 P3 P4 P5 Idle

FCFS

Avg TAT 6.80 · WT 3.80 · RT 3.80 · CPU 100%



P1 0–4 → P2 4–7 → P3 7–8 → P4 8–10 → P5 10–15

SJF

Avg TAT 6.20 · WT 3.20 · RT 3.20 · CPU 100%



P1 0–4 → P3 4–5 → P4 5–7 → P2 7–10 → P5 10–15

SRTF

Avg TAT 6.00 · WT 3.00 · RT 2.80 · CPU 100%



P1 0–2 → P3 2–3 → P1 3–5 → P4 5–7 → P2 7–10 → P5 10–15

Round Robin (q = 2)

Avg TAT 7.60 · WT 4.60 · RT 2.40 · CPU 100%



P1 0–2 → P2 2–4 → P3 4–5 → P1 5–7 → P4 7–9 → P5 9–11 → P2 11–12 → P5 12–15

Priority (non-preemptive)

Avg TAT 7.00 · WT 4.00 · RT 4.00 · CPU 100%



P1 0–4 → P2 4–7 → P4 7–9 → P3 9–10 → P5 10–15

Preemptive Priority

Avg TAT 7.00 · WT 4.00 · RT 3.40 · CPU 100%



P1 0–1 → P2 1–4 → P1 4–7 → P4 7–9 → P3 9–10 → P5 10–15

ALGORITHM	AVG TAT	AVG WT	AVG RT	CPU UTIL
FCFS	6.80	3.80	3.80	100.00%
SJF	6.20	3.20	3.20	100.00%
SRTF	**6.00**	**3.00**	2.80	100.00%
Round Robin	7.60	4.60	**2.40**	100.00%
Priority	7.00	4.00	4.00	100.00%
Preemptive Priority	7.00	4.00	3.40	100.00%

BEST TAT/WT

SRTF improves on SJF because P3 arrives with a shorter remaining time at t=2 and preempts P1.

BEST RESPONSE

Round Robin gives every process an early first turn; RT 2.40 beats SRTF's 2.80.

UTILIZATION TIE

No idle gaps: busy = elapsed = 15, so every algorithm reports **100%**.

IMPLEMENTATION & VERIFICATION

How the project is built

Framework-free scheduling logic, a thin Flask API and two rendering modules.

08

PROJECT FILES

- `app.py` — routes, validation, JSON responses.
- `scheduler/algorithms.py` — six pure-Python schedulers.
- `scheduler/metrics.py` — formulas, averages, best-algorithm helper.
- `templates/index.html` — single-page interface.
- `static/js/main.js` — process CRUD and API calls.
- `static/js/gantt.js` — custom Gantt and tooltips.
- `static/js/charts.js` — Chart.js comparison bars.
- `tests/test_algorithms.py` — 57 automated tests.
- `docs/` — README, function guide, walkthrough, viva and test cases.

API ENDPOINTS

METHOD	PATH	PURPOSE
GET	/	Serve the page
POST	`/api/schedule`	One algorithm
POST	`/api/compare`	All six + winners

Validation failures return HTTP 400 with a clear `error` string. Successful calls return HTTP 200 JSON.

AUTOMATED VERIFICATION

`python -m pytest -q` → **57 passed**.

- All six algorithms on a shared sample.
- SRTF and Preemptive Priority preemption and ties.
- Round Robin expiry ordering and quantum edges.
- Idle gaps, single process, simultaneous arrivals.
- API success and validation failures.
- Static assets and index page.

BROWSER VERIFICATION

The page was driven in headless Chrome:

- 6 algorithm cards and correct card selection.
- SRTF and Preemptive Priority Gantt charts.
- Comparison: 6 rows, 4 best cards, 3 charts.
- Add / duplicate error / edit / delete.
- Empty-list inline error.
- Mobile width 390 px: no page overflow.
- 0 severe console errors.

Limitations: single core; whole-number time units; no I/O, dispatcher latency or context-switch overhead; no aging; at most 100 processes; browser state is not persisted. The application is a teaching simulation and never controls the real OS scheduler.

Viva summary

This project is a Flask-served, browser-based simulator that runs FCFS, SJF, SRTF, Round Robin, non-preemptive Priority and Preemptive Priority scheduling in pure Python. It returns a merged Gantt timeline and CT/TAT/WT/RT metrics as JSON, and visualises the results with a custom Gantt chart and Chart.js comparison charts. It is a teaching simulation, not a real OS scheduler.